

Java OOP Laboratory Workbook

Standard Object-Oriented Implementations: Inheritance, Method Overriding, Interfaces & Execution Tracing

PRACTICAL 01 Single Inheritance & Method Overriding

Problem Statement: Design a superclass `BaseClass` declaring an integer attribute `value = 24` and method `displayMessage()`. Extend it via subclass `ChildAssignment`, override the method to demonstrate dynamic method dispatch and superclass member access, and execute from `main()`.

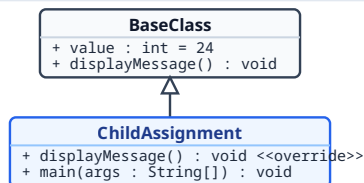
STEP-BY-STEP PROCESS

- **Define Superclass:** Create `BaseClass` with member variable `int value = 24` and display method.
- **Extend Subclass:** Define `ChildAssignment` extends `BaseClass` using Java's `extends` keyword.
- **Apply Method Overriding:** Use `@Override` to redefine `displayMessage()` in the child class.
- **Instantiate & Test:** Create an instance of `ChildAssignment` in `main()` and invoke the overridden method.

ALGORITHM

1. Start program execution in `main()`.
2. Instantiate subclass `ChildAssignment` `obj = new ChildAssignment()`.
3. Inherit and bind superclass field `value = 24`.
4. Invoke overridden method `obj.displayMessage()` dynamically.
5. Print the concatenated value followed by the confirmation line.
6. Stop.

UML CLASS DIAGRAM



SOURCE CODE

```
// Superclass definition
class BaseClass {
    int value = 24;

    void displayMessage() {
        System.out.println("This is the superclass method.");
    }
}

// Subclass demonstrating single inheritance and method overriding
public class ChildAssignment extends BaseClass {
    @Override
    void displayMessage() {
        System.out.println("The value of inherited member is: " + value);
        System.out.println("Method successfully overridden in subclass.");
    }

    public static void main(String[] args) {
        ChildAssignment obj = new ChildAssignment();
        obj.displayMessage();
    }
}
```

SAMPLE OUTPUT

The value of inherited member is: 24
Method successfully overridden in subclass.

Problem Statement: Implement a hierarchical inheritance scheme with root class `Animal` and two derived classes: `Bird` and `AquaticAnimal`. Demonstrate shared base class logic and specialized child behaviors.

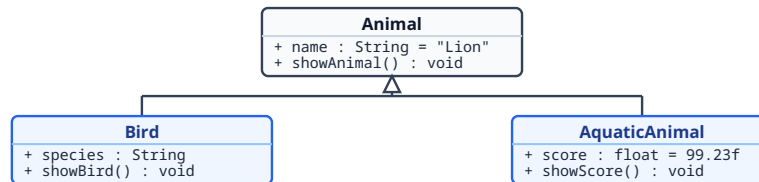
STEP-BY-STEP PROCESS

- **Base Superclass:** Declare `Animal` with field `name = "Lion"` and method `showAnimal()`.
- **Branch 1 Subclass:** Define `Bird` extends `Animal` with specialized attribute and method.
- **Branch 2 Subclass:** Define `AquaticAnimal` extends `Animal` with score attribute and rank method.
- **Driver Execution:** Instantiate derived class and invoke both inherited and specialized methods.

ALGORITHM

1. Start program execution in `HierarchicalDemo.main()`.
2. Instantiate subclass `AquaticAnimal` `sea = new AquaticAnimal()`.
3. Invoke specialized child method `sea.showScore()`.
4. Invoke inherited superclass method `sea.showAnimal()`.
5. Repeat invocation to verify reusable superclass state.
6. Stop.

UML CLASS DIAGRAM



SOURCE CODE

```

class Animal {
    String name = "Lion";

    void showAnimal() {
        System.out.println(name + " is my favorite animal.");
    }
}

// First branch extending Animal
class Bird extends Animal {
    String species = "Parrot, Sparrow";

    void showBird() {
        System.out.println("Bird species include: " + species);
    }
}

// Second branch extending Animal
class AquaticAnimal extends Animal {
    float score = 99.23f;

    void showScore() {
        System.out.println("Performance score: " + score + " (1st Rank)");
    }
}

public class HierarchicalDemo {
    public static void main(String[] args) {
        AquaticAnimal sea = new AquaticAnimal();
        sea.showScore();
        sea.showAnimal();
        sea.showAnimal();
    }
}
  
```

SAMPLE OUTPUT

```

Performance score: 99.23 (1st Rank)
Lion is my favorite animal.
Lion is my favorite animal.
  
```

Problem Statement: Declare an interface `NumericContract` with a constant `CONSTANT_VAL = 23` and method signature `displayValue()`. Implement this interface contract in class `InterfaceBasicDemo`.

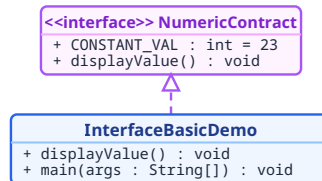
STEP-BY-STEP PROCESS

- **Declare Interface:** Create `NumericContract` with public static final constant and abstract method.
- **Class Realization:** Declare class `InterfaceBasicDemo` implements `NumericContract`.
- **Implement Method:** Provide public concrete definition for `displayValue()`.
- **Driver Execution:** Instantiate implementing class and verify interface member resolution.

ALGORITHM

1. Start driver class `InterfaceBasicDemo`.
2. Instantiate object `obj` of class `InterfaceBasicDemo`.
3. Call implemented method `obj.displayValue()`.
4. Retrieve interface constant `CONSTANT_VAL`.
5. Print formatted output message to the standard console.
6. Stop.

UML CLASS DIAGRAM



SOURCE CODE

```
// Interface contract
interface NumericContract {
    int CONSTANT_VAL = 23; // implicitly public static final
    void displayValue();
}

// Class realizing the interface contract
public class InterfaceBasicDemo implements NumericContract {
    @Override
    public void displayValue() {
        System.out.println("The value from interface contract is: " + CONSTANT_VAL);
    }

    public static void main(String[] args) {
        InterfaceBasicDemo demo = new InterfaceBasicDemo();
        demo.displayValue();
    }
}
```

SAMPLE OUTPUT

The value from interface contract is: 23

Problem Statement: Java prevents multiple inheritance of classes to prevent ambiguity. Demonstrate how multiple inheritance is achieved using multiple interfaces: `SystemConfig` and `PartitionConfig` implemented by class `MultipleInheritanceDemo`.

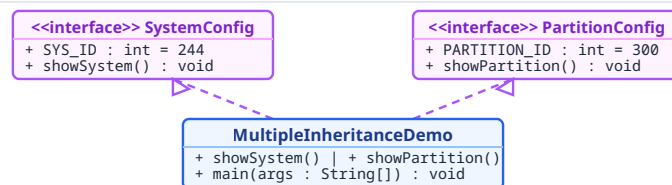
STEP-BY-STEP PROCESS

- **First Interface:** Define `SystemConfig` with constant `SYS_ID = 244` and method `showSystem()`.
- **Second Interface:** Define `PartitionConfig` with constant `PARTITION_ID = 300` and method `showPartition()`.
- **Multiple Realization:** Declare class implementing both interfaces separated by comma.
- **Implement Methods:** Satisfy both abstract method contracts.

ALGORITHM

1. Start driver class `MultipleInheritanceDemo`.
2. Instantiate object `obj` of class `MultipleInheritanceDemo`.
3. Call `obj.showSystem()`; print `SYS_ID`.
4. Call `obj.showPartition()`; print `PARTITION_ID`.
5. Stop.

UML CLASS DIAGRAM



SOURCE CODE

```

interface SystemConfig {
    int SYS_ID = 244;
    void showSystem();
}

interface PartitionConfig {
    int PARTITION_ID = 300;
    void showPartition();
}

// Multiple inheritance realized through multiple interfaces
public class MultipleInheritanceDemo implements SystemConfig, PartitionConfig {
    @Override
    public void showSystem() {
        System.out.println("Multiple inheritance - System ID: " + SYS_ID);
    }

    @Override
    public void showPartition() {
        System.out.println("Multiple inheritance - Partition ID: " + PARTITION_ID);
    }

    public static void main(String[] args) {
        MultipleInheritanceDemo demo = new MultipleInheritanceDemo();
        demo.showSystem();
        demo.showPartition();
    }
}
  
```

SAMPLE OUTPUT

```

Multiple inheritance - System ID: 244
Multiple inheritance - Partition ID: 300
  
```

Problem Statement: Demonstrate interface inheritance where derived interface SubInterface extends parent interface SuperInterface. Realize both contracts in class InterfaceInheritanceDemo.

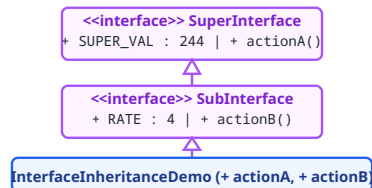
STEP-BY-STEP PROCESS

- **Parent Interface:** Define SuperInterface declaring constant SUPER_VAL = 244 and method actionA().
- **Child Interface:** Declare SubInterface extends SuperInterface adding RATE = 4 and actionB().
- **Implementing Class:** Implement both via InterfaceInheritanceDemo implements SubInterface.
- **Testing:** Instantiate class and verify access to all inherited and implemented members.

ALGORITHM

1. Start driver class InterfaceInheritanceDemo.
2. Instantiate object obj of class InterfaceInheritanceDemo.
3. Print inherited constant obj.SUPER_VAL.
4. Call method obj.actionA(); print message.
5. Print newline separator.
6. Call method obj.actionB(); print rate message.
7. Stop.

UML CLASS DIAGRAM



SOURCE CODE

```

// Base interface contract
interface SuperInterface {
    int SUPER_VAL = 244;
    void actionA();
}

// Sub-interface extending base interface
interface SubInterface extends SuperInterface {
    int RATE = 4;
    void actionB();
}

public class InterfaceInheritanceDemo implements SubInterface {
    @Override
    public void actionA() {
        System.out.println("Value from super interface is: " + SUPER_VAL);
    }

    @Override
    public void actionB() {
        System.out.println("Rate percentage from sub interface is: " + RATE + "%");
    }

    public static void main(String[] args) {
        InterfaceInheritanceDemo demo = new InterfaceInheritanceDemo();
        System.out.println(demo.SUPER_VAL);
        demo.actionA();
        System.out.println();
        demo.actionB();
    }
}
  
```

SAMPLE OUTPUT

```

244
Value from super interface is: 244

Rate percentage from sub interface is: 4%
  
```

Problem Statement: Illustrate multi-interface inheritance where a composite interface `CompositeContract` extends three independent interfaces: `Alpha`, `Beta`, and `Gamma`. Realize all contracts in class `CompositeDemo`.

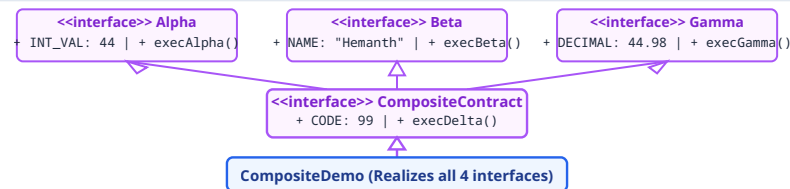
STEP-BY-STEP PROCESS

- **Base Interfaces:** Define `Alpha` (`int`), `Beta` (`String`), and `Gamma` (`double`) with abstract methods.
- **Composite Interface:** Declare `CompositeContract` extends `Alpha`, `Beta`, `Gamma` adding member `CODE = 99`.
- **Realize:** Class `CompositeDemo` implements `CompositeContract` implementing all 4 methods.
- **Driver Execution:** Print state variables and trigger all methods.

ALGORITHM

1. Start driver class `CompositeDemo`.
2. Instantiate object demo of class `CompositeDemo`.
3. Print constants in order: `NAME`, `INT_VAL`, `DECIMAL_VAL`, `CODE`.
4. Invoke all four implemented methods: `execAlpha()`, `execDelta()`, `execBeta()`, `execGamma()`.
5. Stop.

UML CLASS DIAGRAM



SOURCE CODE

```

interface Alpha {
    int INT_VAL = 44;
    void execAlpha();
}

interface Beta {
    String NAME = "Hemanth";
    void execBeta();
}

interface Gamma {
    double DECIMAL_VAL = 44.98;
    void execGamma();
}

// Composite interface extending multiple base interfaces
interface CompositeContract extends Alpha, Beta, Gamma {
    int CODE = 99;
    void execDelta();
}

public class CompositeDemo implements CompositeContract {
    @Override public void execAlpha() { System.out.println(INT_VAL); }
    @Override public void execBeta() { System.out.println(NAME); }
    @Override public void execGamma() { System.out.println(DECIMAL_VAL); }
    @Override public void execDelta() { System.out.println(CODE); }

    public static void main(String[] args) {
        CompositeDemo demo = new CompositeDemo();
        System.out.println(demo.NAME);
        System.out.println(demo.INT_VAL);
        System.out.println(demo.DECIMAL_VAL);
        System.out.println(demo.CODE);
        demo.execAlpha();
        demo.execDelta();
        demo.execBeta();
        demo.execGamma();
    }
}
  
```

SAMPLE OUTPUT

```

Hemanth
44
44.98
99
44
99
Hemanth
44.98
  
```

Problem Statement: Construct a hybrid architecture where subclass `HybridChild` inherits superclass `ParentUnit` while implementing interface `TaskContract`. Trace constructor chaining order and method overrides.

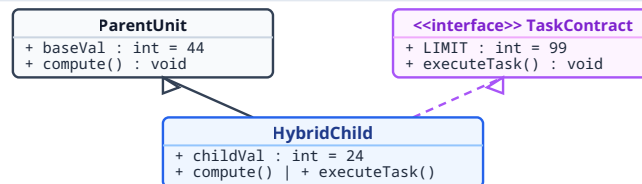
STEP-BY-STEP PROCESS

- **Base Class:** Define `ParentUnit` with constructor printing `baseVal = 44` and method `compute()`.
- **Interface:** Declare `TaskContract` with constant `LIMIT = 99` and method signature `executeTask()`.
- **Hybrid Class:** Declare `HybridChild` extends `ParentUnit` implements `TaskContract` with constructor.
- **Driver Execution:** Trigger new `HybridChild()` and verify constructor invocation hierarchy.

ALGORITHM

1. Start driver class `HybridDemo`.
2. Execute new `HybridChild()`: JVM executes super-constructor `ParentUnit()` first; prints 44.
3. JVM executes subclass constructor `HybridChild()`; prints 24.
4. Call overridden method `child.compute()`; prints $44 + 200 = 244$.
5. Call implemented method `child.executeTask()`; prints $24 + 20 = 44$.
6. Stop.

UML CLASS DIAGRAM



SOURCE CODE

```

class ParentUnit {
    int baseVal = 44;

    ParentUnit() {
        System.out.println(baseVal); // Printed during super-constructor invocation
    }

    void compute() {
        System.out.println(baseVal + 9);
    }
}

interface TaskContract {
    int LIMIT = 99;
    void executeTask();
}

// Hybrid child extending class and implementing interface
class HybridChild extends ParentUnit implements TaskContract {
    int childVal = 24;

    HybridChild() {
        // super() is automatically invoked first
        System.out.println(childVal);
    }

    @Override
    void compute() {
        System.out.println(baseVal + 200);
    }

    @Override
    public void executeTask() {
        System.out.println(childVal + 20);
    }
}

public class HybridDemo {
    public static void main(String[] args) {
        HybridChild child = new HybridChild();
        child.compute();
        child.executeTask();
    }
}
  
```

SAMPLE OUTPUT

```

44
24
244
44
  
```

Problem Statement: Extend the hybrid pattern to achieve full multiple inheritance by deriving class `CompoundChild` from superclass `SuperDevice` while concurrently implementing multiple interfaces: `PrimaryService` and `AuditService`.

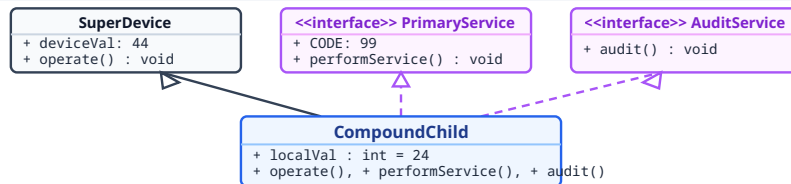
STEP-BY-STEP PROCESS

- **Superclass:** Define `SuperDevice` with constructor printing `deviceVal = 44` and method `operate()`.
- **Interfaces:** Define `PrimaryService` (with `performService()`) and `AuditService` (with `audit()`).
- **Compound Subclass:** Declare `CompoundChild` extends `SuperDevice` implements `PrimaryService`, `AuditService`.
- **Execution:** Instantiate `CompoundChild` and verify constructor chain and method dispatch.

ALGORITHM

1. Start driver class `CompoundHybridDemo`.
2. Instantiate `CompoundChild` `obj = new CompoundChild()`.
3. JVM executes super-constructor `SuperDevice()`; prints 44.
4. JVM executes subclass constructor `CompoundChild()`; prints 24.
5. Call `obj.audit()`; print completion message.
6. Call `obj.performService()`; print calculated result (44).
7. Call `obj.operate()`; print overridden calculation (244).
8. Stop.

UML CLASS DIAGRAM



SOURCE CODE

```

class SuperDevice {
    int deviceVal = 44;

    SuperDevice() {
        System.out.println(deviceVal);
    }

    void operate() {
        System.out.println(deviceVal + 9);
    }
}

interface PrimaryService {
    int CODE = 99;
    void performService();
}

interface AuditService {
    void audit();
}

// Hybrid class inheriting superclass and realizing multiple interfaces
class CompoundChild extends SuperDevice implements PrimaryService, AuditService {
    int localVal = 24;

    CompoundChild() {
        System.out.println(localVal);
    }

    @Override
    void operate() {
        System.out.println(deviceVal + 200);
    }

    @Override
    public void performService() {
        System.out.println(localVal + 20);
    }

    @Override
    public void audit() {
        System.out.println("multiple inheritance is completed");
    }
}

public class CompoundHybridDemo {
    public static void main(String[] args) {
        CompoundChild obj = new CompoundChild();
        obj.audit();
        obj.performService();
        obj.operate();
    }
}
  
```

SAMPLE OUTPUT